

2. RMSE (Root Mean Squared Error)

Definition

RMSE is the square root of MSE

$$\text{RMSE} = \sqrt{\text{MSE}}$$

****Simple Explanation**

- Same as MSE But converted back to original units
- Easier to understand than MSE

Why RMSE?

MSE is in squared units,

3. Mean Absolute Error

MAE is the ****average** of absolute error

Key Idea

- All errors are treated equally

Difference from MSE

- MAE -> Linear Penalty
- MSE -> Penalizes Large Errors More

Interpretation

- Lower MAE -> Better Model

4. R² Score

Definition

R² Tells how much variance in data is explained by the model

Formula

$$R^2 = 1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$$

Simple Explanation

- Measures how well the model fits the data.
- Think of it like "accuracy" for regression.

Range

- 1 -> Perfect Model
- 0 -> Very Poor Model
- Negative -> Worse than average Model

Interpretation

- Higher R^2 -> Better Model

5.Adjusted R^2

Problem with R^2

R^2 always increases when we add more features,even if they are useless.

Solution: Adjusted R^2

Adjusted R^2 fixes this by **penalizing unnecessary features**

Formula

$$\text{Adjusted } R^2 = 1 - (1 - R^2)((n-1)/(n-p-1))$$

Where:

- n = number of samples
- p = number of features

Mini Summary Table

Metric	What it Measures	Best Value
MSE	Average Squares Error	Lower
RMSE	Average Error (Real Units)	Lower
MAE	Average Absolute Error	Lower
R^2		

##Step 1: Import Libraries

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
#Step 2: Load dataset
```

```
data = fetch_california_housing()
```

```
X = data.data
```

```
y = data.target
```

```
#Step 3: Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
#Step 4: Train model
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
#Step 5: Predict
```

```
y_pred = model.predict(X_test)
```

```
#Step 6: Evaluate metrics
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("MSE:", mse)
```

```
print("RMSE:", rmse)
```

```
print("MAE:", mae)
```

```
print("R2 Score:", r2)
```

```
#Step 7: Adjusted R2
```

```
n = X_test.shape[0]
```

```
p = X_test.shape[1]
```

```
adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
```

```
print("Adjusted R2:", adjusted_r2)
```

```
MSE: 0.555891598695242
```

```
RMSE: 0.7455813830127748
```

```
MAE: 0.5332001304956983
```

```
R2 Score: 0.5757877060324526
```

```
Adjusted R2: 0.5749637928613576
```

Introduction to Classification (Machine Learning)

What is Classification?

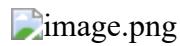
Classification is a supervised learning technique in Machine Learning where the goal is to predict a category or label for a given input data.

Instead of Predicting a Continuous Value (like salary or house price in regression),
Classification

Sigmoid Function

To Convert the linear output into a probability, Logistic Regression uses the Sigmoid Function.

Formula:



Probability Interpretation

After Applying the sigmoid Function, we get Probability:

$P(Y=$

Decision Boundary

The Decision Boundary is the Threshold that separates the two classes in Logistic Regression.

Condition

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load dataset
df = pd.read_csv("C:/Users/harish/Downloads/student_data.csv")

# Features (X) and Target (y)
X = df[['Hours_Studied']]
y = df['Pass']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
model = LogisticRegression()

# Train model
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Predict new student
new_student = pd.DataFrame({
    'Hours_Studied': [6]
})
```

```
prediction = model.predict(new_student)
print(y_pred)
print("Prediction (0=Fail, 1=Pass):", prediction[0])

[1 0]
Prediction (0=Fail, 1=Pass): 1
```

K-Nearest Neighbors (KNN)

The Number of nearest neighbors to consider

Nearest

How KNN Works

When a New Data Point arrives:

1. Calculate the distance between the raw point and all training data points.
2. Select the K nearest neighbors.
3. Count the class labels of those neighbors.
4. Assign the class that appears most frequently.

Distance Metric

KNN Identifies the Nearest Neighbors by calculating the distance between data points.

The Most Commonly used Distance Metric is Euclidean Distance.

Euclidean Distance Formula

THE DISTANCE FORMULA

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

!

where

- d = Distance between Two Points
- (X1,y1) =
- (X2,y2) = Second Point

Simple Explanation

Why Distance is Important in KNN

KNN Predicts a class by finding the nearest neighbors.

the distance metric helps determine:

- Which Points are closest
- Which Neighbors should Participate in Voting.

Choosing the value of k

the value of k plays a crucial role in the performance of the KNN algorithm.

choosing a very small or very large value of k can negatively affect prediction accuracy.

Case 1: Small K

Example 1

$K=1$

The Model Looks at only one Nearest Neighbor.

Advantage

- Captures local patterns effectively
- Can model complex decision boundaries.

Disadvantage

- Very Sensitive to Noise and outliers.
- Single incorrectly labeled point can lead to a wrong prediction.

Example: Suppose the nearest neighbor is mistakenly labeled as Pass instead of Fail.

Case 2: Large K

Example

$K=10$

The Model Considers Many Neighbors before making a prediction.

Advantage'

- More Stable Predictions
- Less Affected by Noisy data.

Disadvantage

- Important Local Patterns may be Ignored.
- Different Classes may get mixed together.

Example

If Most nearby students failed but many distant students passed, a large K may incorrectly predict Pass.

Result

Leads to Underfitting

The Model Becomes too simple and may miss important patterns.

Visual Understanding

Small K	Large K
Sensitive to Noise	Stable Predictions
Complex Decision Boundary	Smooth Decision Boundary
Overfitting Risk	Underfitting Risk

Case 3: Best Choice of K

A Common Rule of thumb is:

$$k \sim \sqrt{n}$$

Where:

- K = Number of Neighbors
- N = Number of Training Samples.

Example:

Given:

N=100

Calculate K:

Curse of Dimensionality in KNN

What is the Curse of Dimensionality?

The Curse of Dimensionality Refers to the problems that occur when the number of features(dimensions) in a dataset becomes very large.

KNN relies on distance calculations to find the nearest neighbors. As the number of dimensions increase

3.Dimensionality Reduction

Reduce the number of Features while preserving important information

Common Techniques:

- Principle Component Analysis (PCA)

- Linear Discriminant Analysis (LDA)

These Techniques Compress data into fewer dimensions.

Gini Impurity

Gini Impurity is a measure used in **Decision Trees** to determine how mixed the classes are within a node

Properties

- $\text{Gini}=0$ -> Pure node
- Higher Gini -> More Mixed Classes.
- Lower Gini -> Better Split
- Used by Cart (Classification and Regression Trees)
- Default Criterion in Scikit-Learn.